S&P 500 Lakehouse

Phase 3 | Gold architecture, BI readiness and FinOps

Project report | 25 October 2026 | Repository: github.com/thehaiderali/dav-medallion-pipeline

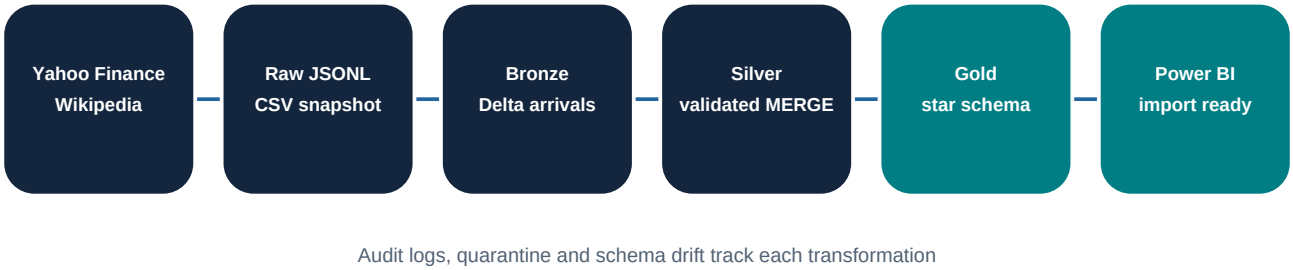

Figure 1. Implemented data path; Power BI is an import specification, not an executed connection.

Executive summary

The pipeline ingests public daily S&P 500 price observations and a fixed company directory. Bronze retains arrivals, Silver validates and deduplicates them, and Gold publishes a dimensional model for trading-activity analysis. Each successful Silver batch triggers Gold refresh automatically. The Gold price fact uses an idempotent MERGE; the sector summary is rebuilt from the current price and company snapshot so sector changes do not leave stale groups.

This report separates implementation from runtime evidence. No Databricks workspace session, billing export, Spark UI capture or Power BI dashboard was supplied for this delivery. The repository contains an executable proof notebook and an import-ready semantic specification; measured production results and the required Spark UI screenshot remain to be collected in a workspace.

Analytical questions

Question	Gold answer
How did trading activity change over time?	Trend of total volume and close-times-volume proxy by trading date.
Which sectors drove activity?	Sector-day volume, proxy value and number of priced companies.
How did an individual ticker trade?	Daily OHLC, adjusted close and volume by company and date.
Was the pipeline healthy?	Gold execution duration/counts, failures, quarantined records and schema drift.

Gold dimensional model

The main fact has one row per ticker and observed trading date. Sector activity is a separate aggregate fact at sector/date grain. The company, sector and date tables are dimensions; all relationships are one-to-many from dimension to fact. Keys are logical contracts and are checked by the proof notebook rather than enforced by Delta constraints.

Table	Key / relationship	Measures or attributes
gold_dim_date	date_key (PK)	Trading date; year, quarter, month, weekday, year-month
gold_dim_sector	sector_key (PK)	Current sector name
gold_dim_company	ticker (PK); sector_key (FK)	Current name and sub-industry
gold_fact_daily_price	date_key + ticker (PK/FKs)	OHLC, adjusted close, volume, dollar-volume proxy
gold_fact_sector_daily	date_key + sector_key (PK/FKs)	Priced-company count, volume, dollar-volume proxy

Relationships

gold_dim_date.date_key to both fact date_key columns; gold_dim_company.ticker to gold_fact_daily_price.ticker; gold_dim_sector.sector_key to gold_dim_company.sector_key and gold_fact_sector_daily.sector_key. The date dimension includes observed trading days; calendar gaps are expected.

Refresh semantics

The existing Bronze-to-Silver notebook now invokes Gold after each successful Silver batch. Gold dimensions and the price fact use hash-guarded MERGE. An identical replay produces zero price/dimension inserts or updates. The sector fact is a complete overwrite each run; this costs a full aggregate scan but handles changed sector assignments cleanly at this project scale. A Gold-only notebook synchronizes all current Silver rows for repair or first deployment. Every Gold step has its own audit row, duration and row counts; the sector step populates rows_aggregated.

Metric meaning

Dollar-volume proxy is close multiplied by share volume. It is not exchange-reported turnover. Company observations count valid priced tickers, not total index membership. Historical prices are classified using the current company snapshot because the source does not provide historical constituents or sector changes.

Read optimization

Delta column pruning and date/company filters support selective BI queries. The pipeline avoids daily OPTIMIZE on roughly 503-row increments; compaction can be scheduled after larger backfills if file counts justify it. No benchmark or physical clustering claim is made without workspace evidence.

Engineering decision log

Decision	Reason and consequence
Raw date slices; no physical Gold partition	One exact-date input per incremental run simplifies replay/backfill. Gold facts are modest; date_key supports filters without tiny daily Delta partitions. Reconsider after file-size/query evidence.
Explicit Silver contracts	try_cast, positivity/range checks and a quarantine table prevent malformed values entering analytics.
Schema drift in Bronze	Additional source fields are retained in rescued_data, added as string columns and logged. Silver accepts known fields and quarantines invalid typed values.
No PII fields	Only public price and company directory data are collected. Source locations and audit IDs contain no person-level data; no user dimension is created.
Idempotent facts	A content hash protects unchanged Gold price rows and preserves their update timestamp across replays.

Spark plan and DAG analysis

The most complex Gold step joins daily price facts to the current company dimension, groups by date_key and sector_key, and writes the sector summary. In a typical Spark physical plan, the dimension can be broadcast if small; otherwise the join may shuffle. Hash aggregation first combines rows within partitions, then an Exchange shuffles by the group keys before the final aggregate and Delta write. Date filters on the price fact can be pushed into the Delta scan for selective queries, although the current complete sector rebuild reads all price dates.

Evidence status: No executed Spark UI DAG screenshot was available. The explanation above is a plan expectation, not an observed stage-level measurement. Run 03_demo_idempotency_and_audit.py in Databricks, open the SQL/Spark UI for SILVER_TO_GOLD_SECTOR_FACT, export the DAG screenshot, and add it to the final submission before claiming this rubric item is complete.

Validation and remaining evidence

The proof notebook asserts Gold price row count equals Silver, unique fact keys, date/company dimension counts, sector-observation totals and volume reconciliation. It also checks that unchanged reruns make no Gold price/dimension updates. Local Python compilation passed; PySpark tests could not be run here because the runtime dependency is unavailable. Only an executed Databricks run can establish serverless Delta behavior. The supplied PDF therefore does not claim a measured DAG, cloud cost or dashboard outcome.

Power BI import specification

Import the five Gold Delta tables through the Azure Databricks connector using a SQL warehouse hostname and HTTP path. Configure the relationships listed on page 2 as active, single-direction, one-to-many. Import pipeline_execution_logs, quarantine_prices and schema_drift_log as separate operational tables; they have different grains from trading observations. Refresh the Power BI model after the Databricks pipeline succeeds. The repository's Gold model guide supplies full DAX definitions.

Suggested measure	Purpose
Traded Volume; Dollar Volume Proxy	Activity totals by date, ticker or sector
Priced Companies; Company Observations	Data coverage and sector breadth
Sector Volume; Sector Dollar Volume Proxy	Sector comparison
Selected Close; Highest Daily High; Lowest Daily Low	Ticker price investigation
Gold Rows Aggregated; Gold Run Seconds	Transformation throughput and runtime
Failed Steps; Quarantined Records	Reliability and data quality

Six recommended visuals

1. KPI cards for volume, proxy value, priced companies and failed steps. 2. Date trend for traded volume and proxy value. 3. Sector bar chart for volume. 4. Single-ticker close-price line with a company slicer. 5. Gold execution duration chart by run timestamp. 6. Quarantine-reason bar chart with schema-drift drill-through.

A close-price line should be filtered to one ticker; summing stock prices across companies is not a meaningful business measure. Audit charts use execution timestamps rather than trading dates. This delivery intentionally contains no Power BI dashboard file, as requested.

FinOps evaluation

No Azure/Databricks usage export or measured daily run duration was supplied. A planning scenario assumes 10 minutes of billable compute per run and an illustrative all-in compute rate of USD 2.00 per hour; it is not a quote or observed spend. Daily schedule: 365 x (10/60) x 2 = **USD 121.67/year**. Hourly schedule: 8,760 x (10/60) x 2 = **USD 2,920/year**, before storage, networking, BI query compute and retries. Hourly operation costs 24 times the assumed daily compute because it runs 24 times as often.

Actual cost should be computed as billable DBU-hours times the applicable rate, plus any separate VM, storage and networking charges for the selected SKU. Azure pricing varies by region and agreement. Export Databricks system.billing.usage or Azure Cost Management records and join job/run IDs to pipeline_execution_logs before replacing this scenario with measured cost. Reduce spend with jobs compute or serverless sizing, auto-termination, incremental aggregation, fewer full scans and compaction only when files warrant it.

Sources: Azure Databricks pricing: https://azure.microsoft.com/en-us/pricing/details/databricks/ ; Azure Databricks billing system table: https://learn.microsoft.com/en-us/azure/databricks/administration-guide/system-tables/billing